

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 1

SUPPORT VECTOR MACHINES

Reading Time	20 minutes
Prior Knowledge	Machine Learning classification, linear and non-linear data, bias and variance
Keywords	Support Vector, Margin, Kernel

In the previous module, we covered two types of classification methods, namely the Naïve Bayes and Decision Trees approach. In this lesson, we will cover another popular method called Support Vector Machines.

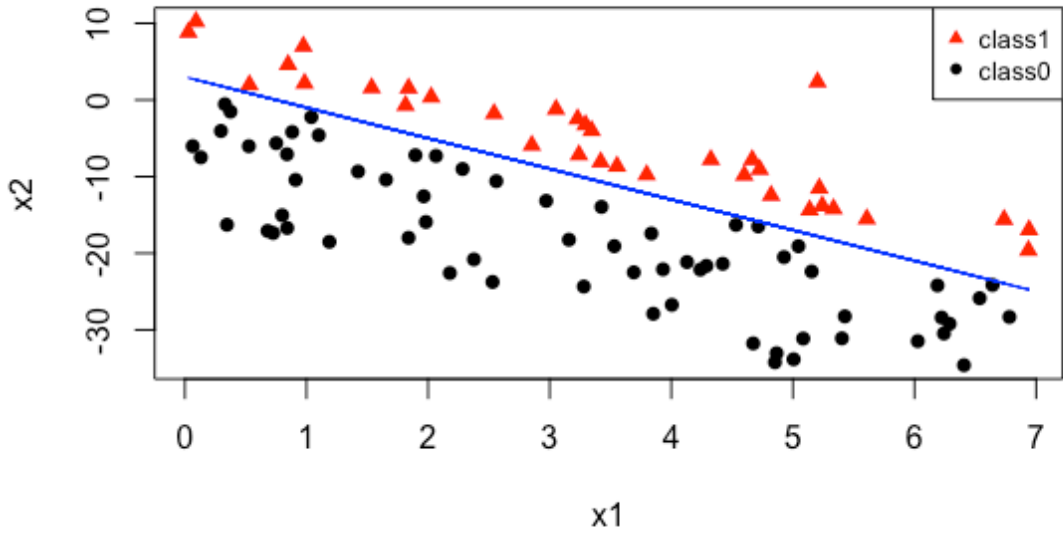
1. Introduction

Support vector machines is an approach to solving classification problems and has grown in popularity since its introduction in the 1990s. Like logistic regression, it is effective for linearly separable data, or data that can be separated by a linear boundary. However, there are hyperparameters that can be used to adjust the boundary to deal with non-linear data. This boundary is what we call the separating hyperplane.

2. Separating Hyperplane

A hyperplane is simply a flat linear plane for any p -dimensional space. An example would be the straight line in a 2-dimensional space as can be seen in fig. 1, i.e., for $p = 2$ dimensions, the hyperplane is a flat $p - 1 = 1$ dimensional subspace. In figure 1 below, the blue line is a hyperplane in a two-dimensional space.

Figure 1: A Linear Boundary Separating a Binary Target Class Data



Mathematically, in this example, the hyperplane can be represented by the equation of a straight line,

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0, \quad (1)$$

hence, in general for p -dimensions,

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0, \quad (2)$$

or,

$$\sum_{j=1}^p \beta_j X_j + \beta_0 = 0. \quad (3)$$

Suppose the equation of the separating hyperplane represented in fig. 1 by the blue line is given by,

$$-4 + 4X_1 + X_2 = 0. \quad (4)$$

Now if we had an observation with (X_1, X_2) coordinates of $(6, -10)$, then substituting this point into (4) would result in,

$$-4 + 4(6) + (-10) = 10 > 0. \quad (1)$$

This observation would be above the hyperplane, along with class 1 (red triangles). The rule for classifying target classes for any observation of (X_1, X_2) would be,

$$-4 + 4X_1 + X_2 > 0 \text{ Then Class 1,} \quad (2)$$

or

$$-4 + 4X_1 + X_2 < 0 \text{ Then Class 0.} \quad (3)$$

The question you may ask is what do we do with an observation that is on the boundary, i.e.,

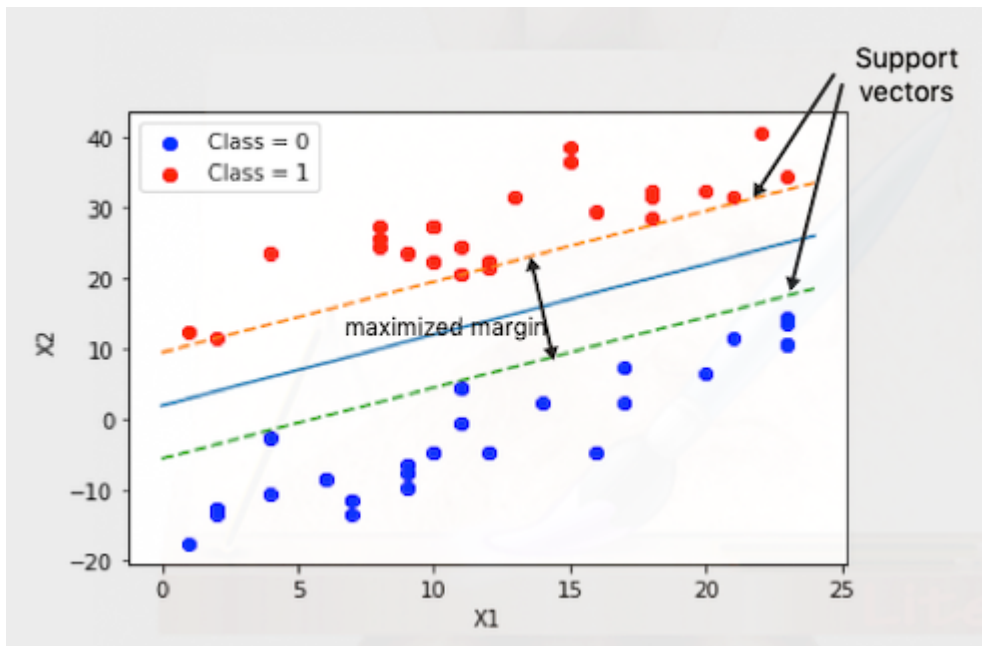
$$-4 + 4X_1 + X_2 = 0 ? \quad (4)$$

To address this, we will need to look at how this hyperplane is derived, and to do this, we look at the Maximal Margin Classifier.

3. Maximal Margin Classifier

You will want to refer to fig. 2 to aid in your understanding of this section. In the example shown in fig. 2, the optimal classifier for two predictors is shown by the blue line. This optimal hyperplane is constructed such that there are equidistant "buffers" on either side shown by the dashed lines. The perpendicular distance between the dashed lines is known as the margin and the optimal hyperplane is achieved when this margin is maximized. Another way of thinking about this is the maximal margin classifier maximizes the distance to the nearest data point. You will notice that the dashed lines touch the data on either side, but it is only necessary to touch the data points on one side before the margin is achieved. These buffers shown by the dashed lines are known as support vectors.

Figure 2: Optimal Boundary Separating Two Classes



For this binary classification problem for predictors X_1 and X_2 , the optimized boundary is obtained by maximizing the margin, M , subject to:

$$\sum_{j=1}^2 \beta_j^2 = 1. \quad (5)$$

For a higher dimension of predictors say p , this is extended to the constraint,

$$\sum_{j=1}^p \beta_j^2 = 1. \quad (4)$$

The constraint in equation (4) is to ensure we look at the perpendicular distance from the j th data point to the hyperplane.

Instead of class 0 and 1 for our binary problem, let us suppose the class labels are -1 and 1 instead, i.e., y_i is -1 or 1 for the i th observation. From (3), a correct classification would be

$$y_i \left(\sum_{j=1}^p \beta_j X_{ij} + \beta_0 \right) > 0, \quad (5)$$

and hence misclassifications as

$$y_i \left(\sum_{j=1}^p \beta_j X_{ij} + \beta_0 \right) \leq 0. \quad (6)$$

A method, called the Perceptron algorithm, determines the weights, β_j , by looking at the misclassifications in (6) and updating the weights in an iterative way. The details of this algorithm will not be covered in this course.

For an extra safeguard, think of the perpendicular distance of the hyperplane to the support vector as D . For this extra safeguard, we can change (5) to be

$$y_i \left(\sum_{j=1}^p \beta_j X_{ij} + \beta_0 \right) > D. \quad (6)$$

One thing to note is that SVMs do not output probabilities of the target class; however, some methods do exist, such as Platt scaling and isotonic regression, to obtain this. Thus far, we have only considered linearly separable data, but often in practice, this may not be the case. The perceptron algorithm does not perform well for non-linearly separable data. This brings us to Hard and Soft margins.

4. Types of Margins

Hard margins are when no misclassifications occur or are allowed. Soft margins are when the number of misclassifications can be specified. A hyperparameter called the regularization parameter, c , specifies the importance of having misclassifications. The higher c , the greater the penalty, i.e. it is very important to have no misclassifications. When c is increased we move towards a hard margin, i.e., no misclassifications. The problem with using a hyperplane that perfectly separates classes in the training dataset is that it may not achieve the same performance for unseen or test data. This is the problem of over-fitting. When c is increased, we are actually taking a more conservative approach by

using a very small margin, i.e., the dashed lines in fig. 2 are much closer to the hyperplane (solid blue line). A smaller c pushes these margins further away since we are allowing the classifier to accommodate more misclassifications. In other words, a small c enables a soft margin, at the cost of some misclassifications. Slack variables are measures of the distance of the misclassified points from the margins, i.e. those points that don't sit perfectly on the correct side of the margin. The parameter c is the constraint on slack variables. It is important to note that some books use c inversely by defining it as the upper bound of the sum of all slack variables. We will use the notation c to keep in line with how it is used in python. Refer to <https://stats.stackexchange.com/q/488107> for the mathematical interpretation. New data introduced into the training set can change the hyperplane; however, it is important to note that only points on the margin and on the wrong side of the hyperplane would affect a change. This quality emphasizes the insensitivity to outliers. The gamma hyperparameter can be adjusted such that nearby or far away points to the decision boundary have an influence. Low gamma takes far away points and vice versa. We finally get to the problem of non-linear data and the approach of SVM to overcome this via kernels.

5. Kernels

Let's watch the video below to get an idea about what a Kernel does.

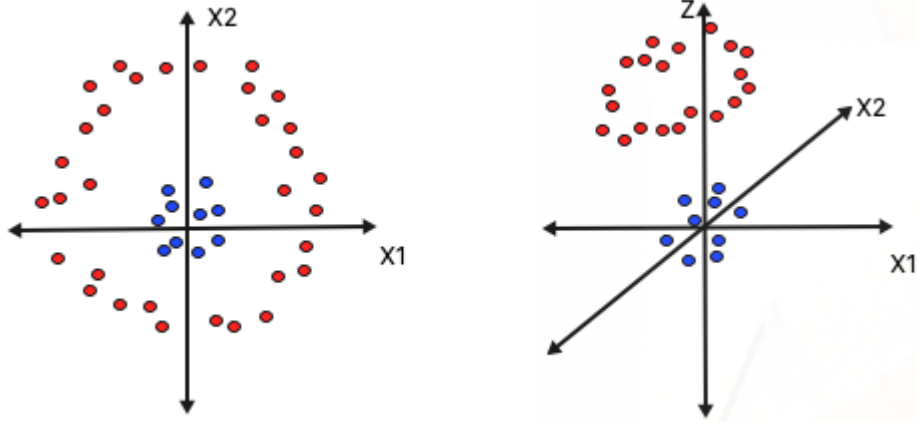
```
In [1]: from IPython.display import VimeoVideo  
  
VimeoVideo("781662374", h="d05a258e6d", width=600)
```

Out[1]:

[Access Video Transcript here](#)

Suppose the target classes are distributed in the feature space between two predictors as shown in the left plot of fig. 3.

Figure 3: Left: Non-linearly separable data. Right: Data projected into an extra dimension Z such that $Z = X_1^2 + X_2^2$.



Using the hard margin approach, a straight line would not split the classes appropriately. An approach may be to add an extra predictor that is a function of the original such that

$$Z = X_1^2 + X_2^2. \quad (7)$$

Projecting the feature space into an extra dimension would show the distribution of the target classes in the right plot of fig. 3. We now see that a plane can separate the target classes efficiently. This mapping into higher dimensions is done by kernels. SVM algorithms have a number of kernels to choose from, such as Polynomial Kernel and Gaussian Radial Basis Function (RBF) to name a few. For a more comprehensive list, please refer to <https://data-flair.training/blogs/svm-kernel-functions/>. One thing to note is that this illustrative example was merely two predictors originally; therefore, with more predictors the optimal transformation is not only difficult find but also to compute. Kernels achieve this in a more memory efficient way. The detailed mathematics behind kernels is not part of the scope of this module, but the interested reader is referred to <https://www.analyticsvidhya.com/blog/2020/10/the-mathematics-behind-svm/>. What we need to understand is that the solution to calculating the optimal hyperplane depends on the dot or inner product of the observations. The inner product for two vectors $a = (a_1, a_2, \dots, a_n)$ and $b = (b_1, b_2, \dots, b_n)$ of equal length n is

$$\langle a, b \rangle = \sum_{i=1}^n a_i b_i. \quad (7)$$

From (7), you may notice that this is 'somewhat' how we obtained the calculation of Z for the example above, i.e.,

$$Z = \sum_{j=1}^2 \langle X_{j_i}, X_{j_{i'}} \rangle = K(X_i, X_{i'}) \quad (8)$$

for two different observations i and i' . We use K to denote the kernel.

In general, a linear support vector can be represented as

$$y(x) = \beta_0 + \sum_i^n \alpha_i \langle X, X_i \rangle \quad (9)$$

for some new point X , where the parameters α_i are estimates over each observation i . This implies that inner products between the new point X and all other observations need to be calculated to estimate α ; however, α is zero for all observations that are not support vectors. We therefore see that determining the α s is made less computationally intensive.

The Gaussian RBF kernel has the below formula instead,

$$K(X_i, X_{i'}) = \exp \left(-\gamma \|X_i - X_{i'}\|^2 \right) \quad (10)$$

where $\|X_i - X_{i'}\|$ is the euclidean distance between observation X_i and $X_{i'}$. This may look familiar to you if you've seen the formula for a Gaussian equation $\exp \left(-\gamma \|X - \mu\|^2 \right)$ for mean μ . The parameter γ is synonymous with the width of the Gaussian distribution or standard deviation, σ , i.e., $\gamma = 1/2\sigma^2$. Hence, this is a hyper parameter provided by the user and a too large γ can lead to over-fitting. The Gaussian Kernel is often used when the original data is non-linearly separable and is sometimes the default kernel in programming languages. It is best to know the default kernel used in the SVM model and experiment with different kernels.

6. Conclusion

In this lesson, we covered the mathematics and methodology behind support vector machines. We looked at the hyperplane and support vectors that form the decision boundaries for separating a binary target class. Methods to possibly overcome non-linearly separable data such as kernels were examined. In the next lesson, we will put SVM into practice with a problem in the finance field.

References

DataFlair. "Kernel Functions-Introduction to SVM Kernel and Example."

<https://data-flair.training/blogs/svm-kernel-functions/>

James, Gareth, et al. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed., Springer, 2021.

Radhika. "The Mathematics Behind Support Vector Machine Algorithm (SVM)." *Analytic Vidhya*, 23 Oct. 2020.

<https://www.analyticsvidhya.com/blog/2020/10/the-mathematics-behind-svm/>

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Blanco, Víctor, et al. "Multiclass Optimal Classification Trees with SVM-splits." *arXiv*, 2021, <https://arxiv.org/abs/2111.08674>.

- **Required pages:** pp. 1–8
- **Estimated reading time:** 45 minutes

2. James, Gareth, et al. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed, Springer, 2021.

- **Required pages:** pp. 367–382
- **Estimated reading time:** 60 minutes

Note: This book is not open access; however, the publisher and authors have made a downloadable copy available online. Due to copyright issues, we cannot provide you with a direct link, but if you google the book's title, you should be able to find and download a copy of it.

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 2

SUPPORT VECTOR MACHINES IN PRACTICE

Reading Time	20 minutes
Prior Knowledge	Classification, Support Vector Machines, Decision Trees.
Keywords	Hyperparameters, slack variable, returns, Receiver Operating Curve (ROC).

In the previous lesson, we covered the methodology of Support Vector Machines. In this lesson, we apply it to a predictive analysis problem.

1. Introduction

In this lesson, we will implement SVM on a trading strategy for the Luxembourg index (LUXXX) based on other country indices as well as technical indicators. The strategy is to take a long/short position when a 0.25% change in LUXXX return is predicted. The long/short position will be based on an upward or downward trending market. Keep in mind that this strategy does not take into account trading costs, but the lesson is designed to show the practicality of SVM in predictive analytics for financial problems.

We will begin by importing the necessary packages.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn import svm
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeClassifier
```

The data containing the weekly closing price of indices can be found in the following csv file along with the dates. We load this data and thereafter view it using the head() command.

Should you store the csv file in the same location as the notebook, then use the uncommented cells below. Use the commented cells instead if you choose to

store the csv in a different location. The "loc" variable is the path to the csv you need to specify.

```
In [2]: # loc = "ENTER YOUR FULL PATH TO LOCATION OF DATA FILE HERE"
# data_df = pd.read_csv(loc+"/MScFE 650 MLF GWP Data.csv")
data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
data_df.head(3)
```

```
Out[2]:
```

	Date	US	UK	Canada	France	Germany	Italy	Japan	Spain	Sweden	Switzerland	Australia	Belgium	Denmark	Finland	Greece	Ireland	Netherlands	Portugal	South Africa	Sweden	Switzerland	United Kingdom	United States
0	1 - Jan 16	139066	235265	368791	106709	100382	100237	106377	5947	...	237137	814963	473810	105084	101040	108406	101137	101830	101675	10101	10101	10101	10101	10101
1	8 - Jan 16	129675	226000	386566	107666	99351	99206	105618	5438	...	220938	71898	38056	104766	100090	103706	100090	100090	100090	100090	100090	100090	100090	100090
2	15 - Jan 16	125186	221580	381289	105348	99342	99406	104088	5180	...	209600	72435	38959	103279	99703	103402	100090	100090	100090	100090	100090	100090	100090	100090

3 rows x 36 columns

The 'Date' column is in a string format; therefore, we convert it to a datetime format.

```
In [3]: # Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

```
/tmp/ipykernel_33/616100313.py:2: UserWarning: Could not infer format, so
each element will be parsed individually, falling back to `dateutil`. To
ensure parsing is consistent and as-expected, please specify a format.
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

Below is where we specify the target column. In our case, it is the LUXXX index. The returns of LUXXX need to be calculated since our strategy is based on the returns over the weeks. The remaining indices are converted to returns instead of their prices. This accomplishes two things, namely it scales the features into a common unit as well as captures movement in the prices instead.

```
In [4]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date",
target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()
```

The cell below calculates the Target column and converts it into a categorical binary variable. We denote 1 for the absolute returns of LUXXX exceeding 0.25% and 0 otherwise. In a downward or bear market, we look to short should the price decrease by more than this threshold and we take a long position in a bear market. The goal here is to predict a percent change more than a threshold. Note that we shift the target column by one week to align the predicted period with the period available for other predictors. The ML algorithm requires the label and feature/predictor values to be captured in the same row or observation.

```
In [5]: # Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF +
"_returns"].shift(-1)

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)
```

It is worth looking at any imbalance in the dataset. The proportion of 34.5% of the target class 1 does not indicate any imbalance.

```
In [6]: # Checking target proportion
round(data_df["Target"].sum() / len(data_df), 4)
```

```
Out[6]: np.float64(0.3452)
```

Now that we have our Target class, we can select which indices contribute to predicting the target class. We use the feature importance tool in `Sklearn` for a Decision Tree to do this. We look at the cumulative importance for the features that make a contribution > 0.

```

In [7]: # Preparing for Train/Test split

NoNaN_df = data_df.dropna()
X = NoNaN_df[ETF_features]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=0)

# USE DECISION TREE FOR FEATURE IMPORTANCE
# Fitting Decision Tree Classification to the Training set

DTree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0,
    max_depth=8,
    min_samples_leaf=30,
    min_samples_split=20,
    # , class_weight=classweight
)
DTree.fit(X_train, y_train)

feature_importances = pd.DataFrame(
    DTree.feature_importances_,
    index=X.iloc[:, :].columns, # don't want the last Target column
    columns=["importance"],
).sort_values("importance", ascending=False)

feature_importances["Cumul_Imp"] = feature_importances.cumsum().iloc[:, :]
# If you want to see only those that explain a % of variation
pct_var = 1.0
feature_importances =
feature_importances[(feature_importances["Cumul_Imp"] <= pct_var)]

```

The cumulative importance shows only 4 indices that contribute, namely MSCI KOREA, DENMARK, FRANCE, and NORWAY. This reduces the features significantly and will also reduce the run time when doing hyperparameter tuning.

In [8]: # PLOT IMPORTANCE

```
# Example data
cols = feature_importances.index
y_pos = np.arange(len(cols))
performance = feature_importances.importance

df2plot = pd.DataFrame(data=performance, index=feature_importances.index)
df2plot["Variable"] = feature_importances.index
df2plot = df2plot[df2plot["importance"] > 0.000]

print(df2plot)

f, ax = plt.subplots(figsize=(6, 5))

sns.barplot(
    x="importance", y="Variable", data=df2plot, label="Variable Ranking",
    color="b"
)

ax.set(xlim=(0, 1), ylabel="", xlabel="importance")

for p in ax.patches:
    width = p.get_width()
    ax.text(
        width + 0.05,
        p.get_y() + p.get_height() / 2.0 + 0.2,
        "{:1.2f}".format(width),
        ha="center",
    )
```

	importance	Variable
MSCI KOREA	0.436123	MSCI KOREA
MSCI DENMARK	0.219591	MSCI DENMARK
MSCI FRANCE	0.196979	MSCI FRANCE
MSCI NORWAY	0.147308	MSCI NORWAY

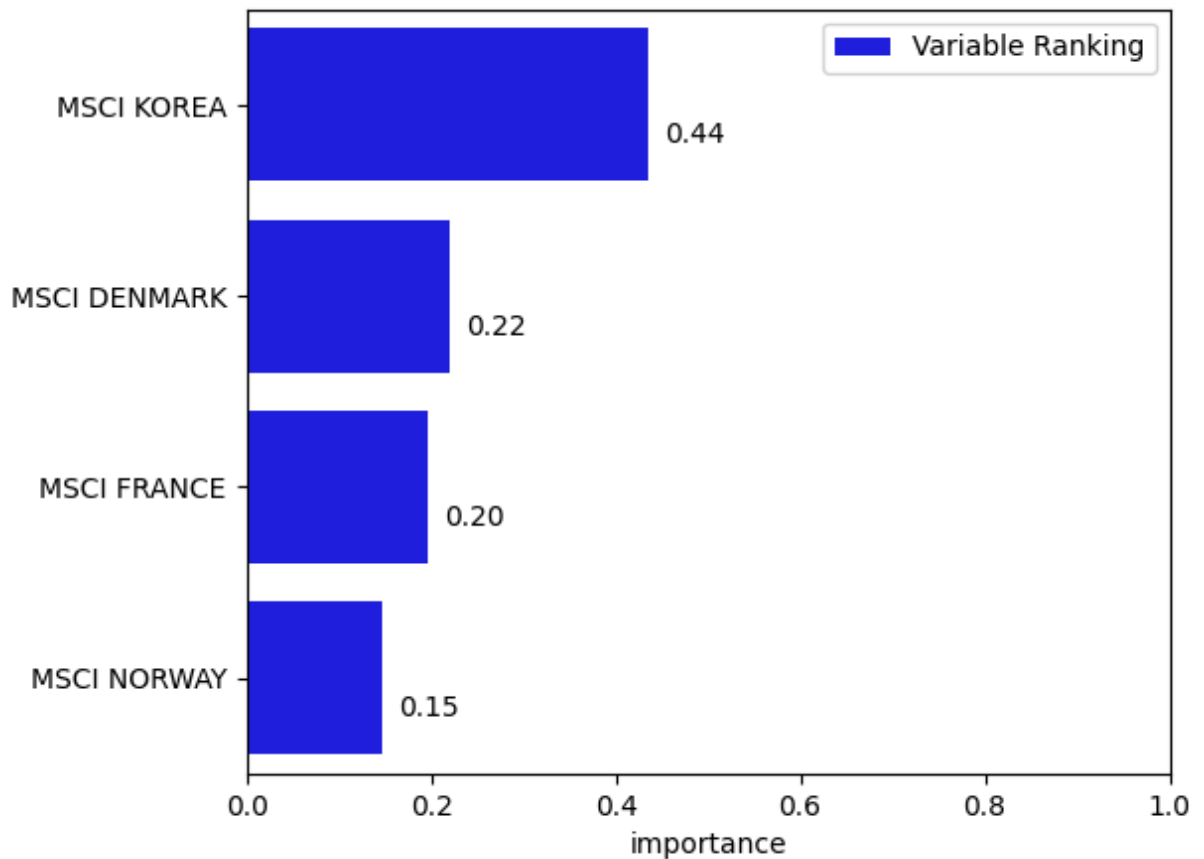


Figure 1: Feature Importance Contribution of Four Predictors

```
In [9]: # Indices to add value in prediction
ETF_ImpFeatures = []
for i in df2plot["Variable"].values:
    ETF_ImpFeatures.append(i)

ETF_ImpFeatures
```

```
Out[9]: ['MSCI KOREA', 'MSCI DENMARK', 'MSCI FRANCE', 'MSCI NORWAY']
```

2. Create Technical Indicators

Among the first technical indicators are the moving average crossovers of short- (SMA_5) and long-term (SMA_15). We use the ratio of (SMA_15/SMA_5) to capture the relationship as one feature and any crossovers. We will rely on the SVM model to capture the relationship between this feature and the target class.

```
In [10]: data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)
```

The next technical indicator we look at is the relative strength index (RSI), which is a momentum oscillator. This indicator measures the magnitude of recent price changes. The target price is usually considered overbought when the RSI > 70%

and oversold when $< 30\%$. This is based on the price change of LUXXX before the week of prediction.

```
In [11]: # shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up",
    "avg_15down"],
    axis=1,
    inplace=True,
)
```

Rate of change is the percentage change in price over a 15-week period. It is a long-term rate of change and is also a momentum indicator. It is also used to spot overbought and oversold instances.

```
In [12]: data_df["RC"] = data_df[target ETF].pct_change(15)
```

We now combine all features, that is the indices' price change as well as the technical indicators.

```
In [13]: # all_feats
ETF_ImpFeatures.append("SMA_ratio")
ETF_ImpFeatures.append("RSI_ratio")
ETF_ImpFeatures.append("RC")
```

We are now in a position to train our model. To begin, we perform the train/test split of the data into an 80/20 split.

```

In [14]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[ETF_ImpFeatures]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)

```

To determine the best kernel and slack variable value, we do a grid search method using `GridSearchCV`. This will explore the specified hyperparameter space for the optimal values over five cross validations.

```

In [15]: # defining parameter range
param_grid = {
    "C": [0.01, 0.1, 1, 10, 100, 1000],
    # 'gamma': [1, 0.1, 0.01, 0.001, 0.0001],
    "kernel": ["rbf", "linear", "poly"],
}

grid = GridSearchCV(svm.SVC(), param_grid, refit=True, verbose=3, cv=5)

# fitting the model for grid search
grid.fit(X_train, y_train)

```

Fitting 5 folds for each of 18 candidates, totalling 90 fits

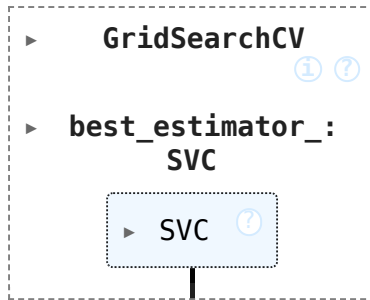
```
[CV 1/5] END .....C=0.01, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=0.01, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=0.01, kernel=rbf;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=0.01, kernel=rbf;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=0.01, kernel=rbf;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=0.01, kernel=linear;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=0.01, kernel=linear;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=0.01, kernel=linear;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=0.01, kernel=linear;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=0.01, kernel=linear;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=0.01, kernel=poly;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=0.01, kernel=poly;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=0.01, kernel=poly;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=0.01, kernel=poly;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=0.01, kernel=poly;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=0.1, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=0.1, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=0.1, kernel=rbf;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=0.1, kernel=rbf;;, score=0.703 total time=
0.1s
[CV 5/5] END .....C=0.1, kernel=rbf;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=0.1, kernel=linear;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=0.1, kernel=linear;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=0.1, kernel=linear;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=0.1, kernel=linear;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=0.1, kernel=linear;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=0.1, kernel=poly;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=0.1, kernel=poly;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=0.1, kernel=poly;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=0.1, kernel=poly;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=0.1, kernel=poly;;, score=0.703 total time=
```

```
0.1s
[CV 1/5] END .....C=1, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=1, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=1, kernel=rbf;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=1, kernel=rbf;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=1, kernel=rbf;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=1, kernel=linear;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=1, kernel=linear;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=1, kernel=linear;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=1, kernel=linear;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=1, kernel=linear;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=1, kernel=poly;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=1, kernel=poly;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=1, kernel=poly;;, score=0.684 total time=
0.1s
[CV 4/5] END .....C=1, kernel=poly;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=1, kernel=poly;;, score=0.676 total time=
0.0s
[CV 1/5] END .....C=10, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=10, kernel=rbf;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=10, kernel=rbf;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=10, kernel=rbf;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=10, kernel=rbf;;, score=0.676 total time=
0.0s
[CV 1/5] END .....C=10, kernel=linear;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=10, kernel=linear;;, score=0.711 total time=
0.0s
[CV 3/5] END .....C=10, kernel=linear;;, score=0.684 total time=
0.1s
[CV 4/5] END .....C=10, kernel=linear;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=10, kernel=linear;;, score=0.703 total time=
0.0s
[CV 1/5] END .....C=10, kernel=poly;;, score=0.711 total time=
0.0s
[CV 2/5] END .....C=10, kernel=poly;;, score=0.737 total time=
0.0s
[CV 3/5] END .....C=10, kernel=poly;;, score=0.684 total time=
0.0s
[CV 4/5] END .....C=10, kernel=poly;;, score=0.703 total time=
0.0s
[CV 5/5] END .....C=10, kernel=poly;;, score=0.676 total time=
```

0.0s
[CV 1/5] ENDC=100, kernel=rbf;;, score=0.737 total time=0.0s
[CV 2/5] ENDC=100, kernel=rbf;;, score=0.737 total time=0.0s
[CV 3/5] ENDC=100, kernel=rbf;;, score=0.684 total time=0.0s
[CV 4/5] ENDC=100, kernel=rbf;;, score=0.703 total time=0.0s
[CV 5/5] ENDC=100, kernel=rbf;;, score=0.676 total time=0.0s
[CV 1/5] ENDC=100, kernel=linear;;, score=0.711 total time=0.0s
[CV 2/5] ENDC=100, kernel=linear;;, score=0.711 total time=0.0s
[CV 3/5] ENDC=100, kernel=linear;;, score=0.684 total time=0.0s
[CV 4/5] ENDC=100, kernel=linear;;, score=0.703 total time=0.0s
[CV 5/5] ENDC=100, kernel=linear;;, score=0.703 total time=0.1s
[CV 1/5] ENDC=100, kernel=poly;;, score=0.711 total time=0.0s
[CV 2/5] ENDC=100, kernel=poly;;, score=0.737 total time=0.0s
[CV 3/5] ENDC=100, kernel=poly;;, score=0.684 total time=0.1s
[CV 4/5] ENDC=100, kernel=poly;;, score=0.703 total time=0.0s
[CV 5/5] ENDC=100, kernel=poly;;, score=0.676 total time=0.0s
[CV 1/5] ENDC=1000, kernel=rbf;;, score=0.737 total time=0.0s
[CV 2/5] ENDC=1000, kernel=rbf;;, score=0.711 total time=0.1s
[CV 3/5] ENDC=1000, kernel=rbf;;, score=0.684 total time=0.0s
[CV 4/5] ENDC=1000, kernel=rbf;;, score=0.622 total time=0.0s
[CV 5/5] ENDC=1000, kernel=rbf;;, score=0.676 total time=0.0s
[CV 1/5] ENDC=1000, kernel=linear;;, score=0.711 total time=0.3s
[CV 2/5] ENDC=1000, kernel=linear;;, score=0.711 total time=0.5s
[CV 3/5] ENDC=1000, kernel=linear;;, score=0.684 total time=0.2s
[CV 4/5] ENDC=1000, kernel=linear;;, score=0.703 total time=0.5s
[CV 5/5] ENDC=1000, kernel=linear;;, score=0.703 total time=0.1s
[CV 1/5] ENDC=1000, kernel=poly;;, score=0.684 total time=0.2s
[CV 2/5] ENDC=1000, kernel=poly;;, score=0.737 total time=0.3s
[CV 3/5] ENDC=1000, kernel=poly;;, score=0.684 total time=0.3s
[CV 4/5] ENDC=1000, kernel=poly;;, score=0.649 total time=0.3s

```
[CV 5/5] END .....C=1000, kernel=poly;, score=0.676 total time=0.3s
```

Out[15]:



The best hyperparameters are specified below. We can see that the best kernel to use for this dataset is the polynomial kernel with a slack variable value of 10.

In [16]:

```
# print best parameter after tuning
print(grid.best_params_)

# print how our model looks after hyper-parameter tuning
print(grid.best_estimator_)
```

```
{'C': 100, 'kernel': 'rbf'}
SVC(C=100)
```

Finally, we can train our SVM model for the specified hyperparameters and compare it to an untuned decision tree classifier.

In [17]:

```
# Train with Tuned SVM
# Create a svm Classifier
clf_tuned = svm.SVC(
    random_state=0,
    C=grid.best_params_["C"],
    kernel=grid.best_params_["kernel"],
    probability=True,
)

clf_tuned.fit(X_train, y_train)

# Fitting Decision Tree Classification to the Training set as well

clf_tree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0,
    max_depth=8,
    min_samples_leaf=30, # DM
    min_samples_split=20,
    # , class_weight=classweight
) # DM
clf_tree.fit(X_train, y_train)
```

Out[17]:

```
▼ DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=8,
min_samples_leaf=30,
min_samples_split=20, random_state=0)
```

From the results below, we see that the SVM model is significantly better than a random guess; however, the tree-based approach performed slightly better with a 4% increase in AUC. All rights reserved WQU WorldQuant University QQQQ

3. Performance

```
In [18]: # Performance

# predicted probabilities generated by sklearn classifier
y_pred_proba = clf_tuned.predict_proba(X_test)
y_pred_probatree = clf_tree.predict_proba(X_test)

# SVM ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
# TREE ROC dependencies
fpr_tree, tpr_tree, _ = roc_curve(y_test, y_pred_probatree[:, 1])

auc = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)
auc_tree = round(roc_auc_score(y_test, y_pred_probatree[:, 1]), 4)

# SVM Model
plt.plot(fpr, tpr, label="SVM, auc=" + str(auc))
# Tree model
plt.plot(fpr_tree, tpr_tree, label="Tree, auc=" + str(auc_tree))
# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```

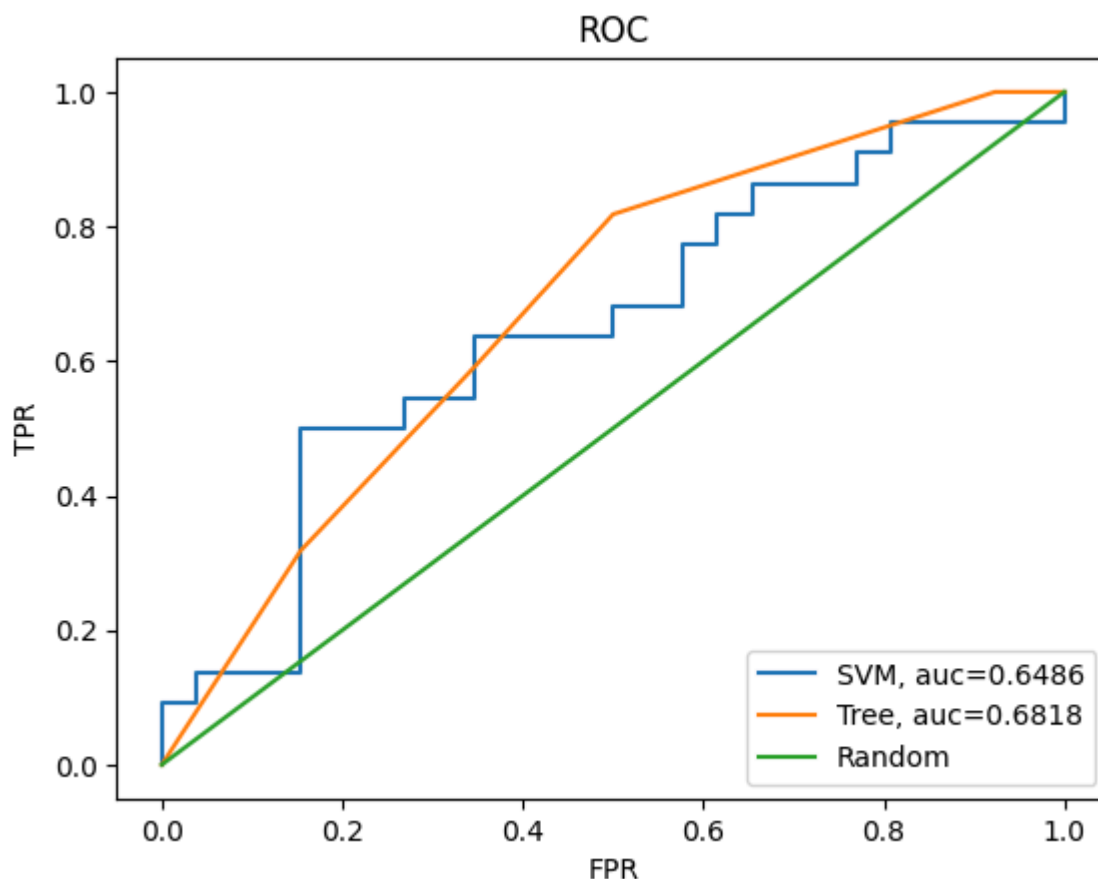


Figure 2. ROC Curve for Three Models

4. Conclusion

This lesson focused on implementing the SVM model in Python on a predictive analytical problem, specifically a trading strategy for long positions of an index. We also included technical indicators as part of the predictors. We found that the SVM provides better results than a random guess, and the results are not too far off from those of a decision tree classifier. As an exercise, you can adjust this notebook to look at a hyperparameter-tuned decision tree classifier as well. In the next lesson, we will look at a model that is very effective on non-linear data, namely an artificial neural network.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Shrikumar, Avanti, et al. "Learning Important Features through Propagating Activation Differences." *arXiv*, 2019, <https://arxiv.org/abs/1704.02685>.

- **Required pages:** 1-3
- **Estimated reading time:** 30 minutes

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 3

NEURAL NETWORKS

Reading Time	20 minutes
Prior Knowledge	Linear and Non-linear data, gradient
Keywords	Activation function, Loss function, Neuron, Layers

---All rights reserved WQU WorldQuant University QQQQ

In the previous lesson, we covered support vector machines, which can handle both linear and non-linear data provided the appropriate kernel is used. In this lesson, we will cover a neural network predictor that is effective for both linear and non-linear data.

1. Neural Network Basics

Neural networks (NNs) are often described as a way to replicate the human brain with a computer. When we think about the process of identifying an image, such as an image of a dog, we first view the image with our eyes thereby sending inputs to the brain. The next step would be the brain analyzing the image based on what we've seen about dogs in the past. Finally, the brain has made a decision/conclusion that this is an image of a dog. To summarize this process, we could have:

Input -> analyze -> decision.

This task is quite simple for us; however, to get a computer to perform the same image recognition task for a dog is not trivial. The process is similar in that the computer undertakes a similar process:

Input -> process/calculations -> output.

Let's look at an example to understand the process of a neural network to discover the relationship between inputs and outputs. We'll consider the simple problem of converting mass from pounds (lbs) to kilograms (kg). Suppose we didn't know the formula for the conversion, and all we had to go on was the observation:

kg	lbs
50	110.23

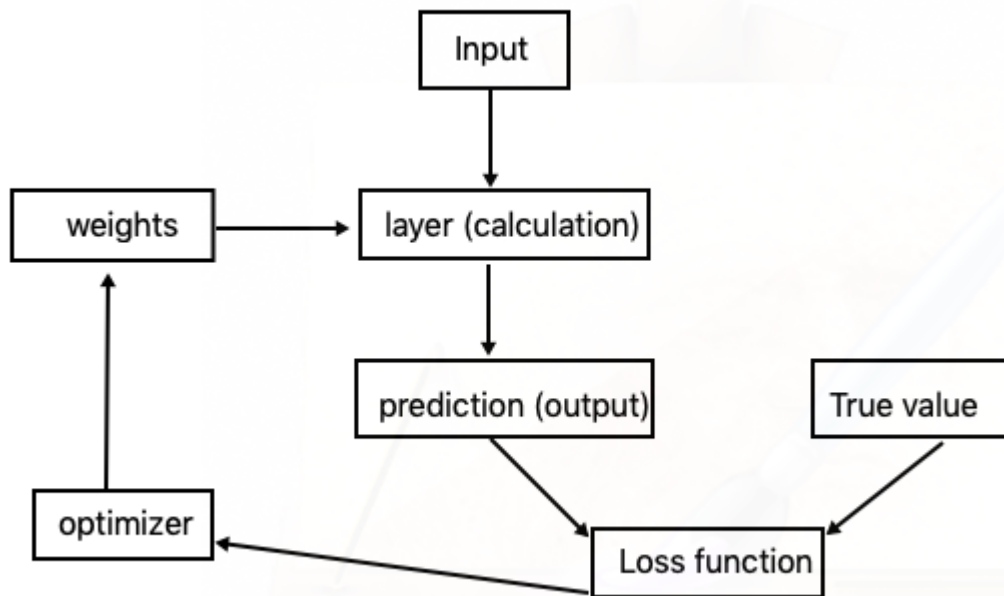
We make a critical assumption here: that the relationship between kg and lbs is linear. I know it's a good guess since we know the formula is $lbs = 2.2046 \times kg$, but often, we don't know the relationship. So we've made our guess that the relationship is $lbs = c \times kg$, where c is the slope. This linear equation is what is called the *activation function* in neural networks. We begin by making a guess about $c = 1.5$, and with this guess, our lbs estimate is $lbs = 1.5(50) = 75$ lbs. we therefore seem to have underestimated by $110.23 - 75 = 35.23$ lbs. Okay, so we guess again, going higher for c based on the feedback from our first guess, and go with $c = 3$, resulting in our second estimate of $50(3) = 150$ lbs. It seems we overestimated this time by 39.77 lbs. We therefore need to go with a value of $1.5 < c < 3$. We do this iteratively until we get to a value of $c = 2.2$, which results in 110 lbs. Pretty close. We can decide this is close enough, therefore settling on $c = 2.2$. We can summarize the iterations below,

Table 1: Summary of Iterations to Find Optimal Slope c .

iteration	c	lbs estimate	(Actual-predicted)
1	1.5	75	35.23
2	3.0	150	-39.77
3	2.5	125	-14.77
4	2.2	110	0.23

The last column provided insight into how far away our guess was from the actual. This is an example of a *loss function* in a NN. The magnitude of the difference between the actual and estimate signals how we need to adjust our next guess for c . Think of the difference being large resulting in our next guess of c being largely different from the previous one. This type of feedback is what we call the *optimizer* in a NN. With this example, we more or less get the workings or flow of a NN. This can be summarized in figure 1 below. You'll notice a 'weight' component in the diagram, which we will get to in the next section. To do this, we will look at the architecture of the NN.

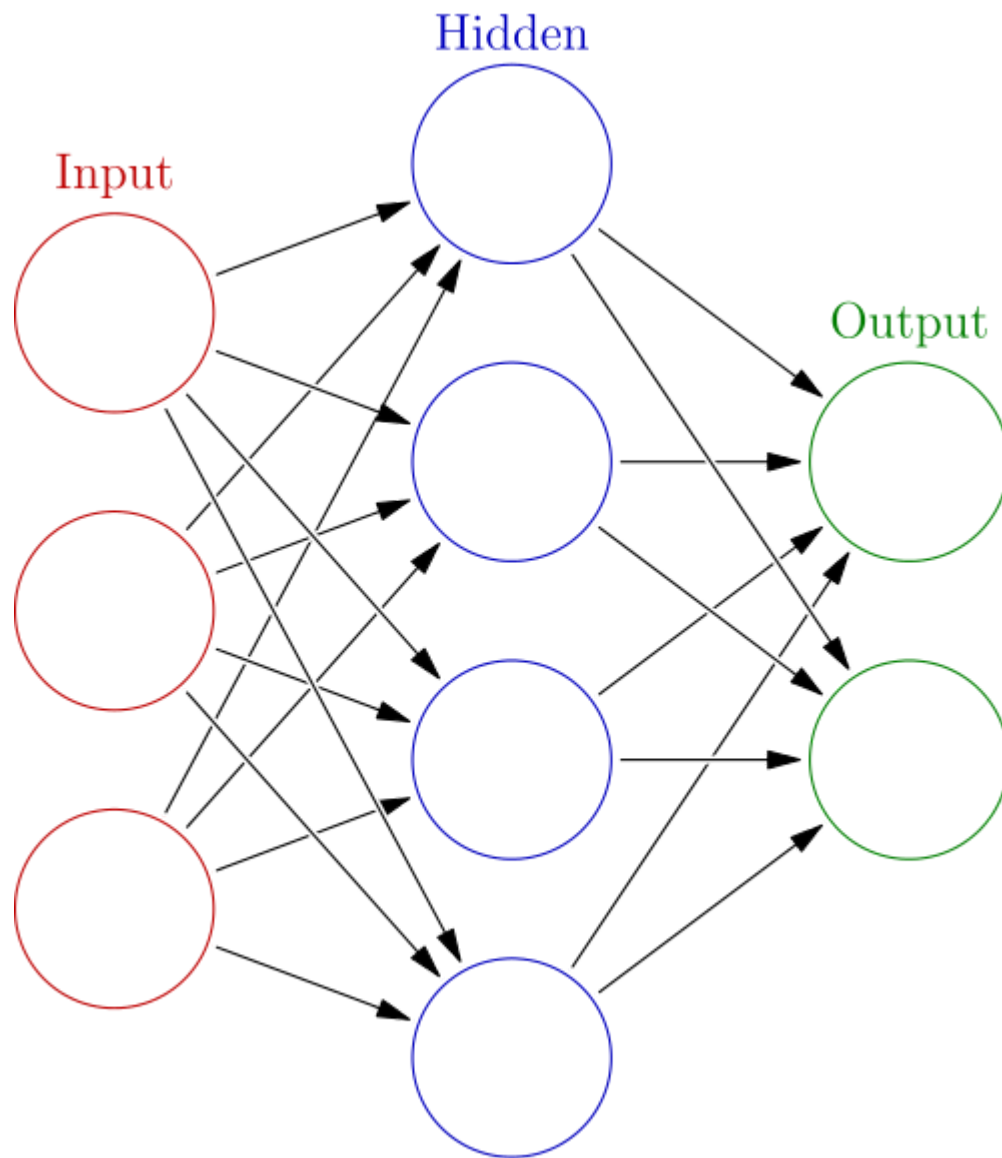
Figure 1: Flow Diagram of Neural Network Operation



2. Network Architecture

A neural network's architecture is shown in figure 2. The hidden layer is where the processing takes place, and do keep in mind that we can have more than one hidden layer, but for now, we keep it simple.

Figure 2: Architecture of Basic Neural Network



Source: [opensource.com](https://opensource.com/article/17/10/fiber-optic-neural-network-art), *How to use an Arduino and Raspberry Pi to Turn a Fiber Optic Neural Network into Wall Art*, June 2022, <https://opensource.com/article/17/10/fiber-optic-neural-network-art>)

If you look at the connections (edges) between the input layer and hidden layer, you will notice all red circles (input nodes) are connected to all the blue circles (Hidden layer nodes). We call this type of connection a *dense* layer. Think of each input node as a feature feeding information into the NN, for example in a credit risk problem, one input node could be Age with another being Salary. Hence, the number of nodes in the input layer is the number of features or predictors. The number of nodes in the output layer will depend on the target variable. If the target were a binary class variable, then one node would be sufficient in the output layer since it returns the probability of being in the target class, e.g., probability of defaulting on a loan. For a regression-type problem, like the lbs to kg conversion discussed earlier, the output node would return the continuous value of the target variable, i.e., lbs.

3. Forward and Backward Propagation

NNs achieve predictions via a process called propagation. At each forward step, there are matrix multiplications taking place. Computers are relatively quick at doing these calculations, and it is worth understanding the process behind it. To begin, we must first identify the dimension of inputs at the start, i.e., at the input layer. We mentioned earlier that the number of nodes at the input layer corresponds to the number of predictors or features. This is the number of columns of the input matrix, and the number of rows corresponds to the number of observations, N say. We thus have a $N \times 3$ matrix of inputs for figure 2. In general, let's denote the number of features as p so figure 2 would have $p = 3$. We shall denote the layers as follows:

1. Input layer: X_0
2. Hidden layer: X_1
3. Output layer: Y

We'll touch on some linear algebra here but nothing too intense other than matrix multiplication. In the forward propagation step moving from X_0 to X_1 , we have the following calculations taking place, which can be represented as matrix multiplication,

$$X_1 = f(X_0 w_0 + b_0), \quad (1)$$

where w_0 is a matrix of $p \times 4$ *weights* since the hidden layer has 4 nodes, and f is a user specified activation function. The term b is a bias term that enables the activation function to shift, which allows more flexibility of the algorithm to fit the data. The activation function, in addition to specifying the linearity/non-linearity of data, also decides on which node to activate by controlling the value of the weights. For instance, suppose Age was not a strong predictor of defaulting on a loan, then the weight for that node would be small. To sequentially move to the output layer, we then have the following calculation,

$$Y = f(X_1 w_1 + b_1) = f(f(X_0 w_0 + b_0) w_1 + b_1), \quad (2)$$

by substituting (1) for X_1 . The dimension of w_1 will be 4×2 since we have 4 nodes in the hidden layer and 2 nodes in the output layer. This sequential process of matrix multiplication results in the following dimension changes:

- $X_0 \times w_0$: $(N \times p) \times (p \times 4) = N \times 4$ (Think of p canceling each other out)
- $X_1 \times w_1$: $(N \times 4) \times (4 \times 2) = N \times 2$

The dimensions of the weights and layers is necessary when specifying the architecture of the NN in a programming language, such as Python. Let's look at two videos that will help illustrate the inputs and outputs at the hidden nodes. Inputs of the hidden nodes can be shown in the video below.

```
In [1]: from IPython.display import VimeoVideo  
  
VimeoVideo("782624672", h="60b0362998", width=600)
```

Out[1]:

[Access Video Transcript here](#)

Outputs of hidden nodes can be show in this video:

```
In [2]: VimeoVideo("782999865", h="2294f6c7e5", width=600)
```

Out[2]:

[Access Video Transcript here](#)

Thus far, we have shown the forward propagation process to achieve outputs; however, it is the backward propagation process that enables the NN to learn and improve. This is the feedback process from the loss function to the optimizer, which eventually tells the algorithm how the weights should be adjusted. Ultimately, it is the weights, and some bias term, that will be adjusted to achieve optimal performance. A great benefit of NNs is that we do not have to restrict ourselves to just one hidden layer, and by increasing the layers, we

increase our chances of solving non-linear data relationships. There is a problem that NNs face with this increase in layers, which is called the *vanishing gradient problem*.

Refer to the video below on how the bias is optimized using derivatives.

```
In [3]: VimeoVideo("785293805", h="a725197abc", width=600)
```

Out[3]:

[Access Video Transcript here](#)

4. Vanishing Gradient

We've briefly mentioned the role of the activation function in section 1. There are various activation functions to choose from with each having their benefits. The reader is directed to (Nwankpa 7) for a brief list of these functions. The sigmoid activation function is a popular function used in binary classification problems. It squashes the output into a small range between 0 and 1. The functional form may look familiar to you if you've worked with a logistic regression model as it is calculated as follows,

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1)$$

What this tells us is that for a large change in x or input, we have a small change in output. But we know that the gradient is given by,

$$\text{Gradient} = \frac{\text{change in output}}{\text{change in input}} \quad (2)$$

hence this implies the gradient tends to a small value for the sigmoid function. The connection of the sigmoid function to the loss function, L say, is that we know our output depends on the activation function. The more layers we have in our network, the more activation functions are required in the process. The idea of the backpropagation from table 1 was to minimize L or the difference in the

last column through feedback. Calculus is very handy when we want to minimize some function f with respect to its input x . The process entails determining the value of x , say x' , that makes the derivative or gradient of $f = 0$.

Backpropagation thus entails finding the derivative of the initial layer by multiplying the derivatives of each layer moving from the final to the initial layer, a process known as the chain rule. Thus, the problem arises where this product tends to zero with many layers, especially with the sigmoid activation function. This prevents optimal updating of the weights and bias.

Improvements to NNs have been made since 2009 to overcome this problem such as better activation functions like ReLu, better weight initialization schemes, and improved optimization schemes. NNs have come a long way since their inception, and with GPUs available, the speed of training these algorithms has been greatly improved.

5. Conclusion

In this lesson, we briefly explained the process of a NN. We mentioned its components such as inputs, hidden and output layers consisting of nodes, the forward and backward propagation process, and the role of the activation, loss, and optimizer in the operation of the algorithm. Neural networks also face problems such as vanishing gradients and overfitting if not tuned correctly. We'll look at an example of this in the next lesson where we train a neural network on a financial problem.

References

Nwankpa, Chigozie. "Activation Functions: Comparison of Trends in Practice and Research for Deep Learning." *arXiv Preprint arXiv:*, 1811.03378, 2018, pp. 5-8, arxiv.org/pdf/1811.03378.pdf.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 4

NEURAL NETWORKS IN FINANCE

Reading Time	25 minutes
Prior Knowledge	Neural Network architecture, Economic data
Keywords	Keras, TensorFlow, Layers, Mean Square Error, Mean Absolute Error

The previous lesson covered the workings and architecture of a neural network. In this lesson, we put this into practice by exposing the reader to additional data preparation and model compilation.

1. Predictive Problem

In this lesson, we will look at predicting monthly returns on the Dow Jones Industrial (DJI) Futures, specifically the Mini DJI Futures with ticker YM=F. The predictors we use are economic indicators from the FRED economic database. You may be familiar with this site from the Financial Data course. We consider five economic indicators, namely:

1. Average hourly earnings of all employees
2. All Employees Manufacturing
3. Producer Price Index by Industry
4. Durable goods New Orders
5. Manufacturers' New Orders: Total Manufacturing

Economic data can be found on this website <https://fred.stlouisfed.org/> and made available using an API key, which you can obtain by signing up with an account. Let's begin by importing the Economic data using the `pandas_datareader` package.

2. Getting the Data

We import the `pandas_datareader` below as well as `pandas` since we will be working with DataFrames.

```
In [1]: import pandas as pd
import pandas_datareader as pdr # Access FRED
```

Now that we've imported the necessary package required to obtain the data from FRED, we specify the API key that we obtained after creating an account. It is quite an easy and quick process to obtain an API key.

```
In [2]: # extract api key: put your key in between the angle brackets < >
# myKey = "xxx"
fred_api_key = "<673681d3d91301ee2979bee9f9e5d204>"
```

We'll use this function created below called `get_fred_data`, which makes importing the data very simple. The input parameters are the list of economic indicator keywords and the date range of the data, i.e., start and end date.

```
In [3]: # Using code from FRED API: Get US Economic Data using Python

def get_fred_data(param_list, start_date, end_date):
    df = pdr.DataReader(param_list, "fred", start_date, end_date)
    return df.reset_index()
```

Below is the list of the economic indicator keywords for the five predictors. These keywords are obtained from the FRED website. See below encapsulated in red in Figure 1 as an example.

Figure 1: Keyword for Manufacturers' New Orders - Total Manufacturing.

A list of indicators used and considered is shown below in the next cell. We import the data and take a glimpse using the pandas `tail(10)` command to view the last 10 records. Note that we consider monthly data and take data from Jan. 2000 to Apr. 2022 inclusively.

```
In [4]: # PCUOMFGOMFG :      Producer Price Index by Industry - Monthly
# BOGZ1FL073164003Q :      Interest Rates and Price Indexes; NYSE Composite
# CES05000000003 :          Average hourly earnings of all employees
# MANEMP:                   All Employees Manufacturing
# DGORDER:                  Durable goods New Orders
# AMTMNO:                   Manufacturers' New Orders: Total Manufacturing
# FEDFUNDS:                 Fed fund rate. Lagging indicator

series = ["CES05000000003", "MANEMP", "PCUOMFGOMFG", "DGORDER", "AMTMNO"]
# get data for series
df = get_fred_data(param_list=series, start_date="2000-01-01",
end_date="2022-05-03")
df.set_index("DATE", drop=True, inplace=True)
print(df.shape)
df.tail(10)
```

(269, 5)

Out[4]:

	CES05000 00003	MANEMP	PCUOMFG OMFG	DGORDER	AMTMNO
DATE					
2021-08-01	30.78	12353	220.114	264287	534721
2021-09-01	30.98	12393	221.869	259192	533538
2021-10-01	31.14	12453	225.478	263534	546544
2021-11-01	31.23	12498	227.109	268650	554883
2021-12-01	31.39	12537	226.470	277487	566241
2022-01-01	31.60	12564	231.495	280925	571817
2022-02-01	31.64	12601	237.245	277036	571624
2022-03-01	31.80	12674	245.404	277554	583312
2022-04-01	31.96	12726	249.560	279157	586866
2022-05-01	32.10	12735	256.396	278740	593328

So now that we have our predictors, let's get the target variable, which is the adjusted close price for the Mini DJI futures. We get this data using the yahoo finance (*yfinance*) library. We import monthly data in keeping with the same timeframe as our predictors for the same time period. We have imported Open, High, Low, Close, Adj Close and Volume; however, we consider only the Adj Close that takes into account corporate actions.

In [5]: `import yfinance as yf`

```
# Mini DJI Futures ticker is YM=F
data = yf.download(tickers="YM=F", start="2000-01-01", end="2022-05-03",
interval="1mo")
# Print data
print(data.shape)
data.tail(10)
```

```
/tmp/ipykernel_4247/2699266512.py:4: FutureWarning: YF.download() has
changed argument auto_adjust default to True
  data = yf.download(tickers="YM=F", start="2000-01-01", end="2022-05-03",
interval="1mo")
```

```
[*****100%*****] 1 of 1 completed
```

(206, 5)

Out[5]:

Price Ticker Date	Close YM=F	High YM=F	Low YM=F	Open YM=F	Volume YM=F
2021-06-01	34399.0	34827.0	32902.0	34570.0	2619173
2021-07-01	34832.0	35072.0	33623.0	34448.0	3242166
2021-09-01	33722.0	35504.0	33478.0	35382.0	3555180
2021-10-01	35704.0	35777.0	33383.0	33752.0	3648899
2021-11-01	34457.0	36446.0	34384.0	35796.0	3671355
2021-12-01	36226.0	36572.0	33928.0	34524.0	3248399
2022-01-01	34997.0	36832.0	33031.0	36304.0	5689330
2022-02-01	33840.0	35752.0	32167.0	34915.0	5115575
2022-03-01	34618.0	35281.0	32305.0	33822.0	3648322
2022-04-01	32882.0	35413.0	32820.0	34702.0	3703705

We have our predictors and target variable in different DataFrames; however, they can be joined/merged using the common date index column. We do this using the pandas merge command. It is also necessary to look at missing values or Nulls in the dataset. This could be for unavailability of data on those dates. Fortunately, there are 39 rows missing for the Average hourly earnings of all employees (CES0500000003) column, which isn't too many. We adopt the simplest approach of dropping them since there aren't too many missing values. We are left with 166 rows.

```
In [6]: # Make time zones non-timezone aware so as to allow the join
df.index = df.index.tz_localize(None)
data.index = data.index.tz_localize(None)
```

```
In [7]: df2predict = pd.merge(df, data["Close"], left_index=True,
right_index=True)
print(df2predict.tail())
print(len(df2predict))
df2predict.isnull().sum()
df2predict = df2predict.dropna()

print("Print rows remaining after removed missing values
{}".format(len(df2predict)))
```

	CES0500000003	MANEMP	PCUOMFGOMFG	DGORDER	AMTMNO	YM=F
2021-12-01	31.39	12537	226.470	277487	566241	36226.0
2022-01-01	31.60	12564	231.495	280925	571817	34997.0
2022-02-01	31.64	12601	237.245	277036	571624	33840.0
2022-03-01	31.80	12674	245.404	277554	583312	34618.0
2022-04-01	31.96	12726	249.560	279157	586866	32882.0

206
Print rows remaining after removed missing values 166

We've done a lot of work to put this dataset together, so let's store it as a csv so we can easily read it in when we want to work with it. Specify the path to store it and the name; in my case, I called it `DJI_FuturesPredict.csv`. When we want to read the data in, we just use the pandas `read_csv` command as in the cell below.

```
In [8]: # Store to csv
#path2copy = "../..data"
df2predict.to_csv("DJI_FuturesPredict.csv", index=True,
index_label="Date")
```

```
In [9]: # read in pre-stored data

df2predict = pd.read_csv("DJI_FuturesPredict.csv")
df2predict.set_index("Date", drop=True, inplace=True)

# Quick check that the data looks familiar.
df2predict.tail()
```

```
Out[9]:
```

	CES0500 000003	MANEMP	PCUOMF GOMFG	DGORDE R	AMTMNO	YM=F
Date						
2021-12-01	31.39	12537	226.470	277487	566241	36226.0
2022-01-01	31.60	12564	231.495	280925	571817	34997.0
2022-02-01	31.64	12601	237.245	277036	571624	33840.0
2022-03-01	31.80	12674	245.404	277554	583312	34618.0
2022-04-01	31.96	12726	249.560	279157	586866	32882.0

3. Data Analysis and Preparation

Neural Networks are sensitive to the scale of the features for accuracy as well as the speed of training. There is no single best way to scale the data as we could use percentage change, min max, or standard scaling, among other suggestions, and it depends on the problem. In this example, we use the min max scaling method from `sklearn`. The formula for this is:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

where,

- x_{min} = minimum feature value
- x_{max} = maximum feature value

```
In [10]: # Feature scaling
from sklearn.preprocessing import MinMaxScaler

# scale features
scaler = MinMaxScaler()
scale_model = scaler.fit(df2predict[series])
df2predict[series] = scale_model.transform(df2predict[series])
df2predict.tail()
```

```
Out[10]:
```

	CES0500 000003	MANEMP	PCUOMF GOMFG	DGORDE R	AMTMNO	YM=F
Date						
2021-12-01	0.952181	0.406547	0.755816	0.852189	0.918290	36226.0
2022-01-01	0.969799	0.416051	0.808957	0.874648	0.940380	34997.0
2022-02-01	0.973154	0.429074	0.869765	0.849243	0.939616	33840.0
2022-03-01	0.986577	0.454769	0.956049	0.852627	0.985920	34618.0
2022-04-01	1.000000	0.473073	1.000000	0.863099	1.000000	32882.0

We're interested in predicting the returns of the DJI Futures. As such, we transform the Adj Close price column to percentage returns instead.

```
In [11]: import numpy as np
# Use percent change instead of actual values
# If all columns are transformed to a % change.
df2predict = df2predict.pct_change()

# Replace infinite values with NaN before dropping
df2predict.replace([np.inf, -np.inf], np.nan, inplace=True)

# % change for just the target column
# df2predict["YM=F"] = df2predict["YM=F"].pct_change()

# Drop any missing values (now includes NaNs from original NaNs and
# replaced infs)
df2predict.dropna(inplace=True)
# If a time shift is needed because of a lag.
# `df2predict['YM=F_shift'] = df2predict['YM=F'].shift(-1)`
# `df2predict.drop('YM=F', axis=1, inplace=True)`

# Glimpse of data
df2predict.head()
```

```
Out[11]:
```

	CES0500 000003	MANEMP	PCUOMF GOMFG	DGORDE R	AMTMNO	YM=F
Date						
2006-05-01	-0.307692	-0.008096	0.590909	0.017665	0.057582	-0.020915
2006-06-01	1.000000	0.003549	0.285714	0.078519	0.044573	0.005274
2006-07-01	0.388889	-0.008840	-0.022222	-0.158060	-0.113893	-0.001512
2006-08-01	0.120000	-0.011060	0.090909	0.015674	0.023180	0.014337
2006-09-01	0.285714	-0.012266	-0.625000	0.381904	0.201838	0.031428

It is a good idea to look at the relation of the predictors to the Adj Close price returns. We can do this visually since there aren't too many predictors considered. To avoid too much clutter in the plots and determine any time lag between the Adj Close returns and predictor, we choose a sample of points, say 3 years or 36 months. The visual below does not show strong correlations between the predictors and returns; however, the decrease in Feb. and Mar. of 2020 is experienced for most predictors as well as the returns.

```
In [12]: import matplotlib.pyplot as plt
import numpy as np
```

```

In [13]: target = "YM=F"
n_pts = 36

for j in np.arange(len(series)):
    # Check the lag

    var = series[j]
    # Define Data
    # choose sample of points to avoid clutter in plot
    df2plot = df2predict.iloc[len(df2predict) - n_pts : len(df2predict),]
# noQA E203
    x = df2plot.index
    data_1 = df2plot[var]
    data_2 = df2plot[target]

    # Create Plot

    fig, ax1 = plt.subplots()

    ax1.set_xlabel("Date")
    ax1.set_ylabel(var, color="red")
    ax1.plot(x, data_1, color="red")
    plt.xticks(rotation=90) # Rotates X-Axis Ticks by 45-degrees
    # Adding Twin Axes

    ax2 = ax1.twinx()
    ax2.plot(x, data_2, color="blue")

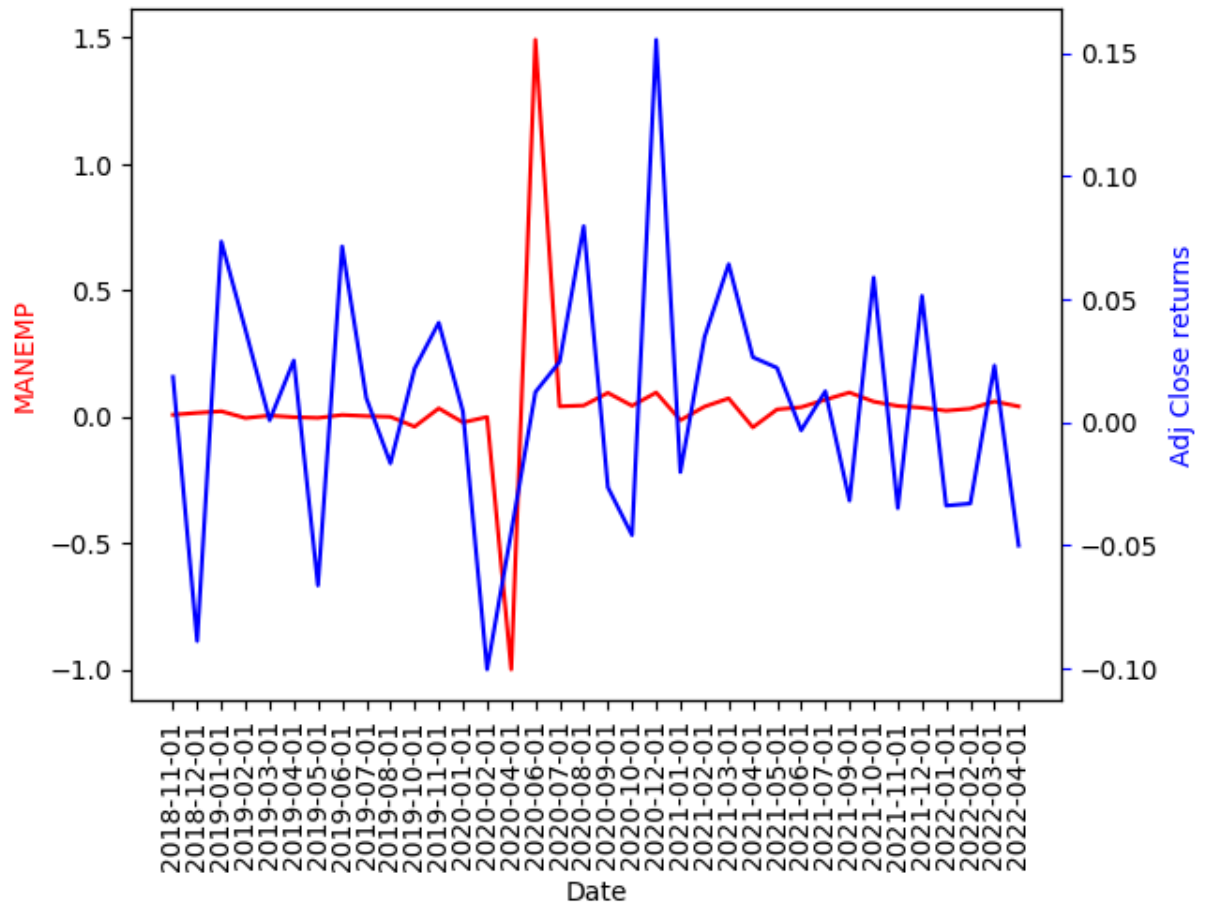
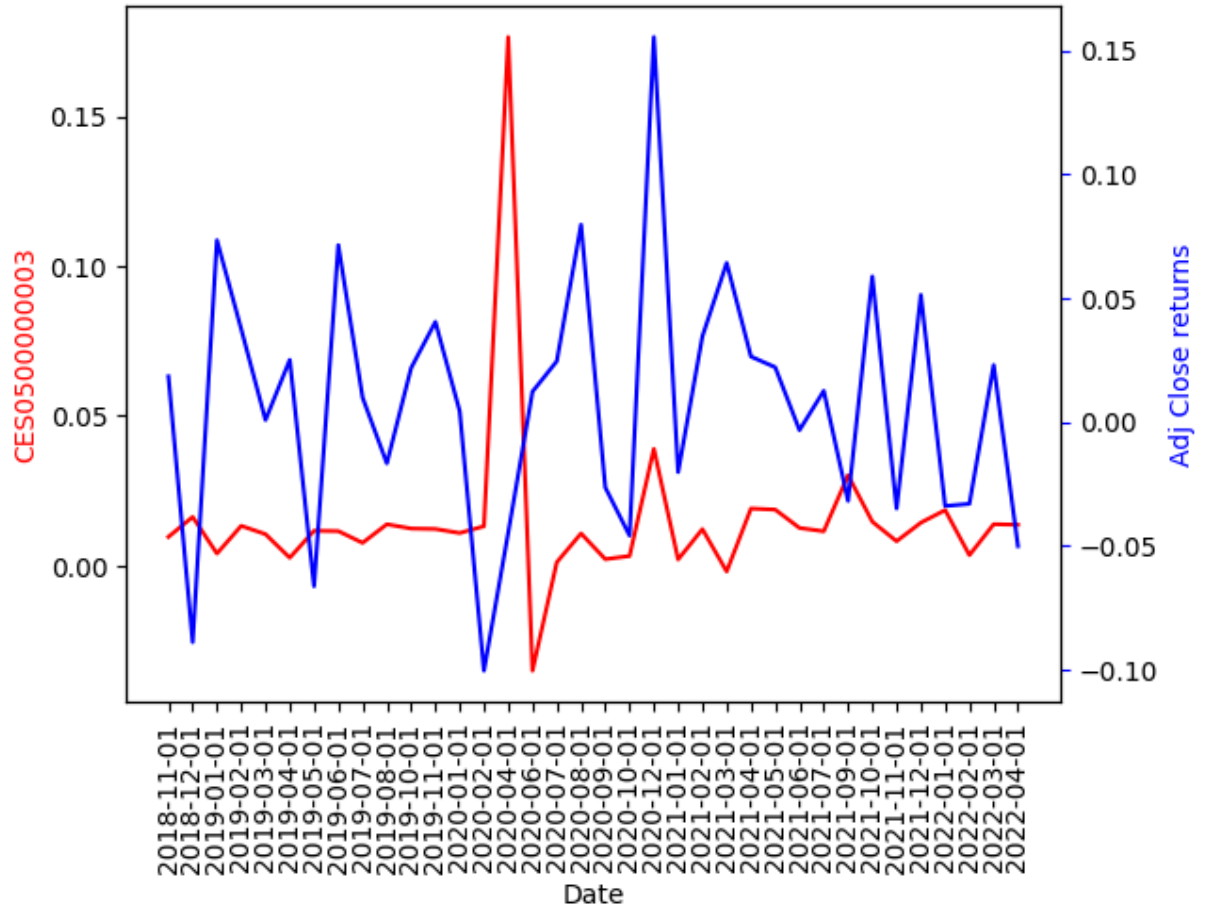
    # Add label

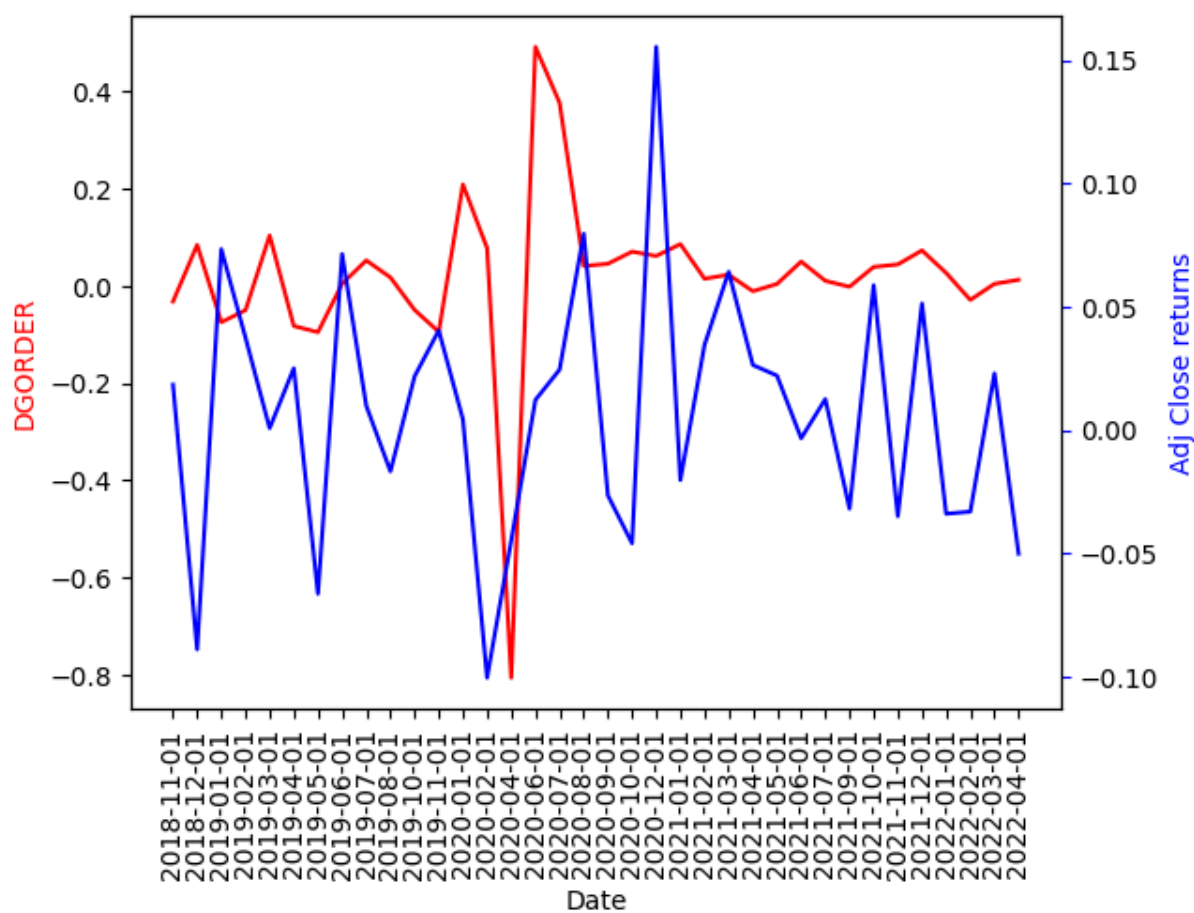
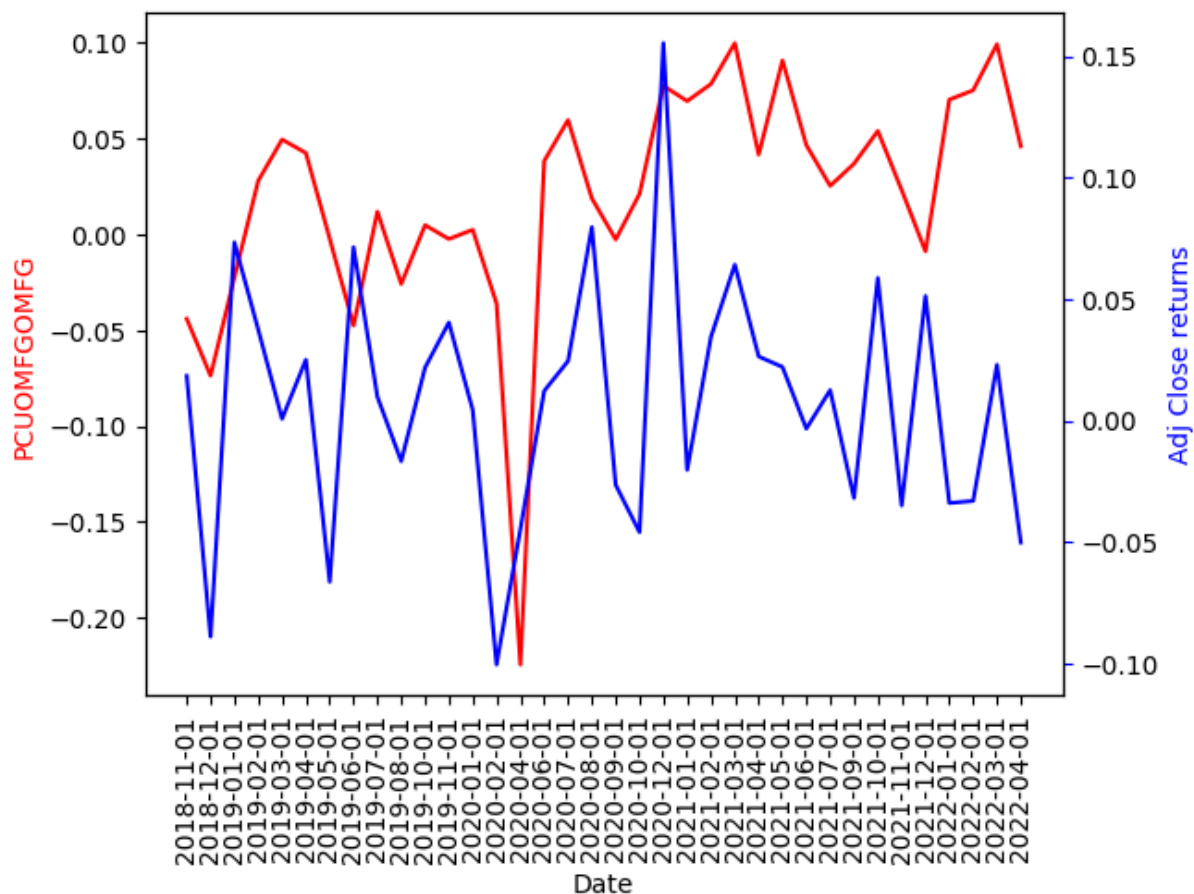
    ax2.set_ylabel("Adj Close returns", color="blue")
    ax2.tick_params(axis="y", color="blue")

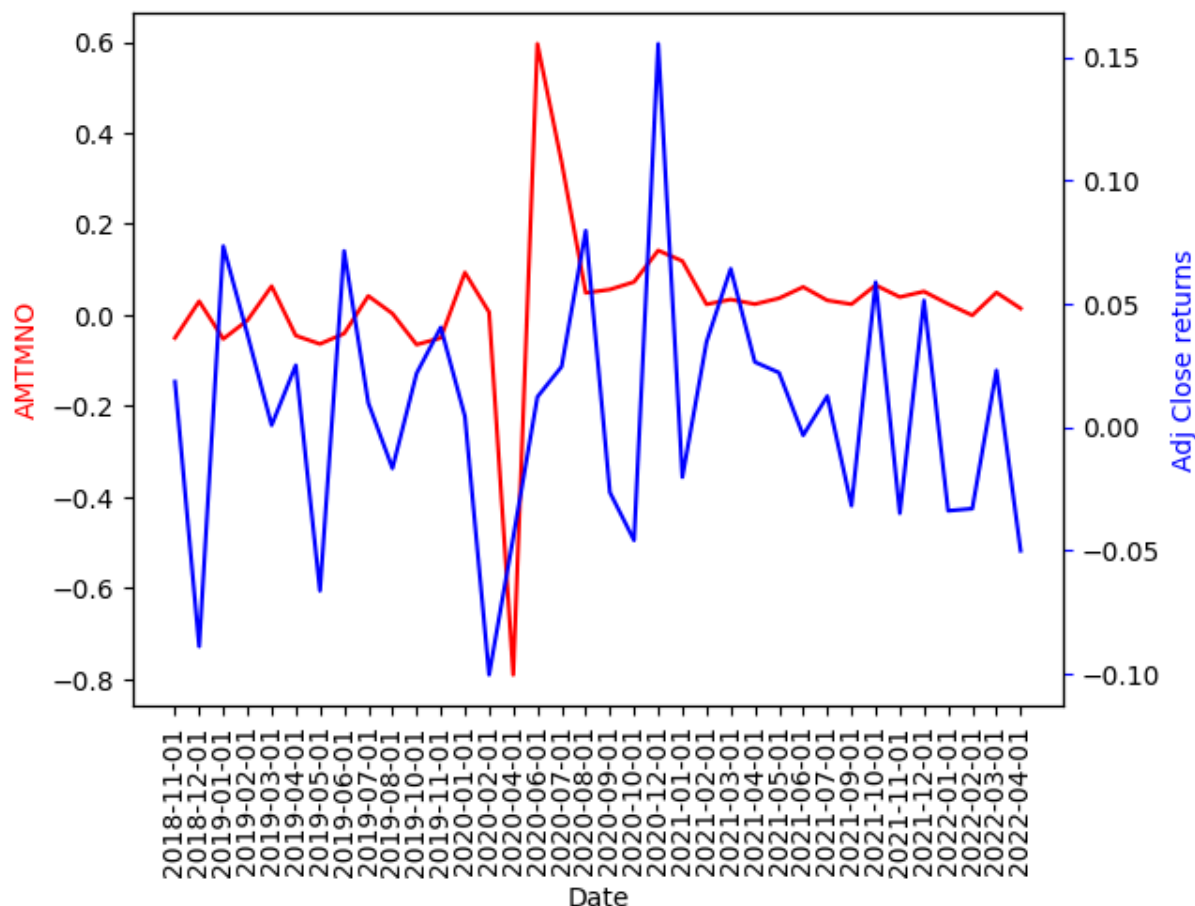
    # Show plot

    plt.show()

```







With that, we are now ready to create our Neural Network using TensorFlow and Keras.

4. Keras and TensorFlow

Neural networks were initially made easily in Python thanks to the Keras library made by Francois Chollet. His book is also a great resource for neural networks and deep learning (Chollet 54); however, it is not open source.

Google created a library called TensorFlow (tf) that did the computations for neural networks efficiently, but it is not the most user-friendly.

Google later released TensorFlow 2 that integrated the Keras API directly and promoted this interface as the default or standard interface for deep learning development on the platform.

This is why we often see `tf.keras` used in deep learning code, which makes it much easier and faster to compile and run these models.

```
In [14]: # Begin creating our model by importing TensorFlow
import tensorflow as tf

print(tf.__version__)
```

2.20.0

We now separate the data into a predictors dataset and target column into x and y respectively. All rights reserved WQU WorldQuant University QQQQ

```
In [15]: X = df2predict[series].values  
y = df2predict["YM=F"].values
```

We perform the necessary train/test split for our data required for predictive modeling. In our case the train/test split is 80/20 and the split is done in a time-ordered manner with recent data being the unseen dataset.

```
In [16]: test_sz = 0.2  
train_sz = int((1 - test_sz) * len(X))  
X_train = X[:train_sz]  
y_train = y[:train_sz]  
X_test = X[train_sz:]  
y_test = y[train_sz:]  
len(X_train), len(X_test)
```

```
Out[16]: (128, 33)
```

In the below cell, we will do a couple of things for the model build. We set a random seed to enable us to replicate this build at any time. We then call the various `tf.keras` objects to put our model together starting with *Sequential*. It is referred to as sequential because we are able to add layers to our NN one after another in sequence. In the simplest example for illustrative purposes, we have only one layer, but we will add onto it later. The `tf.keras.Dense` object should sound familiar from Module 4, Lesson 3, in that we are specifying that all edges connect to the subsequent nodes. The parameter of 1 in the `Dense(1)` object specifies that the output is a single continuous value, i.e., our predicted DJI Futures return.

The `model.compile` command is where we specify the loss function and optimizer. In our example below, we use the mean absolute error as our loss function to gauge how far away our predictions are from actuals. For a list of supported loss functions, see ("Keras Loss Functions").

The optimizer is SGD or stochastic gradient descent, but Adam is another popular option. The performance metric after each iteration is the mean absolute error, but we could also use mean square error. For a list of supported metrics, see ("Keras Metrics").

You'll notice there are specified arguments in the optimizer such as learning rate and momentum. The learning rate specifies the amount of change to the model relative to the change in error. Of course, we would naturally want our model to learn quickly; however, we should bear in mind that a large value for the learning rate could result in learning from a less than optimal set of weights too quickly, thus leading to non-convergence. This could result in an unstable learning process. A learning rate that is too small would increase the training time, possibly leading to the algorithm getting stuck. Typically, learning rate values between 0.01 to 1 are used. Momentum can speed up the training

process to converge with fewer epochs. Values of momentum between 0.9 to 0.99 are often used.

Epochs are the number of times the algorithm will work through the training set. The more epochs, the more time the model needs to train. With limited computing power this needs to be given some thought as the time required could increase significantly. Another reason we can't just increase epochs to as high a number as possible is that this could also lead to over-fitting. We choose 10 here for a simple illustrative purpose but will increase it later to see the change in results. Note that small learning rates would require more epochs and vice versa.

The batch size is the sample size sent through at one time to be propagated through the network. The error gradient calculation is based on these samples that pass through. This would of course need to be smaller than the total number of observations in our training data. The stochastic gradient descent optimizer updates the weights based on the error gradient. Common sizes are in increments of 8. You don't want the batch size being too large a proportion of the number of training set observations. Refer to ("Batch Size Tutorial") for how to choose batch size.

It is worth noting that the batch size and epochs work with the entire training set (80% of observations) available. All observations of the training set are sampled in sizes of the batch size and fed through the algorithm at different times.

Now that we've specified the hyperparameters needed, we can compile and train the model with model.fit.

```
In [17]: # Build the model
# tf.keras: The Keras API integrated into TensorFlow 2

tf.random.set_seed(42) # first we set random seed
model = tf.keras.Sequential([tf.keras.layers.Dense(1)]) # The output
layer

# We compile the model specifying loss, and optimizer.
model.compile(
    loss=tf.keras.losses.mae, # Los function is MAE, mean absolute error.
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ), # stochastic Gradient descent Optimizer
    metrics=["mae"],
) # performance metric is MAE

model.fit(X_train, y_train, epochs=10, batch_size=8) # epoch and batch
size specified
```

```

Epoch 1/10
16/16 ————— 1s 3ms/step - loss: 0.0867 - mae: 0.0867
Epoch 2/10
16/16 ————— 0s 3ms/step - loss: 0.0673 - mae: 0.0673
Epoch 3/10
16/16 ————— 0s 3ms/step - loss: 0.0566 - mae: 0.0566
Epoch 4/10
16/16 ————— 0s 3ms/step - loss: 0.0528 - mae: 0.0528
Epoch 5/10
16/16 ————— 0s 3ms/step - loss: 0.0510 - mae: 0.0510
Epoch 6/10
16/16 ————— 0s 3ms/step - loss: 0.0487 - mae: 0.0487
Epoch 7/10
16/16 ————— 0s 3ms/step - loss: 0.0445 - mae: 0.0445
Epoch 8/10
16/16 ————— 0s 3ms/step - loss: 0.0394 - mae: 0.0394
Epoch 9/10
16/16 ————— 0s 3ms/step - loss: 0.0380 - mae: 0.0380
Epoch 10/10
16/16 ————— 0s 3ms/step - loss: 0.0362 - mae: 0.0362

```

```
Out[17]: <keras.src.callbacks.history.History at 0x7a23507216a0>
```

After training our model, we can look at performance. We use the mean absolute error and mean square error since we are dealing with a continuous target variable. We see that both these metrics are below 5% and 0.3%, respectively.

```

In [18]: # performance
preds = model.predict(X_test)
mae = tf.keras.losses.MeanAbsoluteError()(y_true=y_test,
y_pred=preds.squeeze()).numpy()
mse = tf.keras.losses.MeanSquaredError()(y_true=y_test,
y_pred=preds.squeeze()).numpy()
mae, mse

```

```
2/2 ————— 0s 34ms/step
```

```
Out[18]: (np.float32(0.051342975), np.float32(0.0057897707))
```

Let's see if we can improve on this result with more layers and different choices of the hyperparameters, such as increasing the number of epochs. We also specify an activation function, namely the *sigmoid* function in this example. We also use the Adam optimizer function instead of the SGD. The SGD optimizer is sensitive to the learning rate, and although it can take many iterations, it can still achieve convergence relatively quickly. Adam is a popular optimizer and is used quite effectively in deep learning algorithms. Unlike SGD, Adam does not use a fixed learning rate as it adapts over iterations.

We see that both metrics, MAE and MSE have decreased with 2 additional layers. Adding more layers ventures into the space of Deep Learning and helps when data is non-linear. Remember that the algorithm becomes susceptible to the vanishing gradient problem as more layers are added, especially when the activation function is the sigmoid function.

In [19]: *# Improve our model. More epochs, added extra layers*

```
tf.random.set_seed(42)
model_1 = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            8, activation="sigmoid", input_shape=(X_train.shape[1],)
        ), # added extra layer
        tf.keras.layers.Dense(4, activation="sigmoid"), # added extra
layer
        tf.keras.layers.Dense(1),
    ]
)
model_1.compile(
    loss=tf.keras.losses.mae, optimizer=tf.keras.optimizers.Adam(),
    metrics=["mae"]
)
model_1.fit(X_train, y_train, epochs=100, batch_size=8, verbose=0)
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Out[19]: <keras.src.callbacks.history.History at 0x7a235016af60>

In [20]: *# performance*

```
preds = model_1.predict(X_test)
mae = tf.keras.losses.MeanAbsoluteError()(y_true=y_test,
y_pred=preds.squeeze()).numpy()
mse = tf.keras.losses.MeanSquaredError()(y_true=y_test,
y_pred=preds.squeeze()).numpy()
mae, mse
```

2/2 ————— 0s 54ms/step

Out[20]: (np.float32(0.037677407), np.float32(0.0023999598))

5. Conclusion

In this lesson, we applied a neural network to predicting the monthly returns of the DJI Futures. We used economic indicators from FRED as predictors and explored ways to import and format this data for a predictive analytics scenario. In setting up the neural network, we discussed several hyperparameters required to compile the network and added more layers in our last attempt. This addition of more layers is necessary when creating a deep learning model.

References

- Chollet, Francois. *Deep Learning with Python*. Manning, 2017.
- "Keras Loss Functions." Keras Loss Functions List, www.tensorflow.org/api_docs/python/tf/keras/losses. Accessed 18 June 2022.

- "Keras Metrics." Keras Metrics List, www.tensorflow.org/api_docs/python/tf/keras/metrics. Accessed 18 June 2022.
- "Batch Size Tutorial." Batch Size Tutorial, www.machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size. Accessed 18 June 2022.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.